44. OWASP Top 10 Pratik Kılavuzu: SQL Injection, XSS, CSRF

Web Güvenliğinin En Tehlikeli 10 Açığını Kod Seviyesinde Kapatmak
Bölüm 8 • Zorluk Derecesi: İleri Seviye • Tahmini Okuma Süresi: 13 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Bir hacker sadece bir yorum kutusuna `<script>` yazarak şirketinizin binlerce kullanıcısının oturumunu nasıl çalabilir? XSS ve CSRF'i nasıl tarihe gömeriz?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 OWASP Top 10, dünyada web uygulamalarına yönelik en yaygın ve yıkıcı 10 güvenlik riskini listeler. - 📌 XSS (Cross-Site Scripting), saldırganın web sayfasına zararlı JavaScript enjekte etmesidir; `dangerouslySetInnerHTML` kullanımından kaçının. - 📌 IDOR (Insecure Direct Object Reference), URL'deki ID'yi değiştirerek başkasının faturasına erişmektir; yetkilendirme kontrolü şarttır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir apartmanda oturduğunuzu düşünün.

1. **XSS:** Kapıdaki zil kutusuna bir saldırgan görünmez zehirli bir iğne yerleştiriyor. Komşunuz zile bastığında zehirleniyor ve anahtarlarını düşürüyor (Kötü niyetli JavaScript'in oturum çerezlerini çalması).
2. **CSRF:** Posta kutunuza sahte bir mektup geliyor: *"Bu zarfı açarsan 1000 TL kazanacaksın!"* Zarfı açtığınız anda arka kapıdaki kurye sizin adınıza bankaya talimat veriyor!
3. **IDOR:** Faturanıza bakmak için web sitesine giriyorsunuz. URL şöyle: `site.com/fatura/101`. Yukarıdaki 101 sayısını silip 102 yazıyorsunuz ve komşunuzun tüm kredi kartı ve harcama dökümü önünüze açılıyor!

OWASP Top 10 işte bu apartmandaki güvenlik açıklarının listesidir. Güvenli bir sistem, kapı numarasını değiştiren birine *"Dur bakalım, bu ev senin mi?"* diye soran sistemdir.

🧠 2. AI Çağında Yeni Zihniyet Modeli

En Kritik 3 OWASP Açığının Çözüm Haritası:

```
1. IDOR (Insecure Direct Object Reference):
   ❌ YANLIŞ: SELECT * FROM invoices WHERE id = req.params.id;
   ✅ DOĞRU: SELECT * FROM invoices WHERE id = req.params.id AND user_id =
req.user.id;

2. XSS (Cross-Site Scripting):
   ❌ YANLIŞ: <div dangerouslySetInnerHTML={{ __html: userComment }} />
   ✅ DOĞRU: <div>{userComment}</div> (React varsayılan olarak HTML'i sanitize eder)

3. CSRF (Cross-Site Request Forgery):
   ❌ YANLIŞ: Basit çerezler (SameSite ayarı yok)
   ✅ DOĞRU: SameSite=Strict ve CSRF Token doğrulaması
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

````
# ROL: Web Uygulama Güvenliği Uzmanı (AppSec)
Aşağıdaki Express API endpoint'i tipik bir IDOR (Yetkisiz Nesne Erişimi) açığı
içermektedir.

## AÇIKLI KOD:
```typescript
app.get("/api/invoices/:id", authenticate, async (req, res) => {
  const invoice = await db.invoice.findUnique({
    where: { id: req.params.id }
  });
  res.json(invoice);
});
````

GÖREV:

1. Bu IDOR açığının bir saldırgan tarafından nasıl istismar edileceğini açıkla.
2. Kodda hem sahiplik kontrolünü (Ownership Check) hem de 404/403 durumlarını doğru yöneten güvenli sürümü yaz.
3. Bu açığı otomatik yakalayacak bir güvenlik test senaryosu (Jest/Supertest) tasarla.
````

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// IDOR Açığı: Kullanıcı giriş yapmış ama faturanın KENDİSİNE ait olduğu kontrol edilmiyor!
app.get("/api/orders/:id", authenticate, async (req, res) => {
  // Saldırgan :id yerine rastgele sayılar deneyerek herkesin siparişini çalar!
  const order = await db.order.findUnique({ where: { id: req.params.id } });
  res.json(order);
});
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```typescript
// IDOR Güvenliği: Çift Koşullu Sorgu ve İzolasyon
app.get("/api/orders/:id", authenticate, async (req: Request, res: Response) => {
  const currentUserId = req.user.id;
  const orderId = req.params.id;

  // 1. Veritabanı sorgusunda hem sipariş ID'si hem de oturumdaki kullanıcı ID'si aranır!
  const order = await db.order.findFirst({
    where: {
      id: orderId,
      userId: currentUserId // SAHİPLİK KONTROLÜ (Ownership Verification)
    }
  });

  // Bilgi sızdırmamak için 403 yerine 404 döneriz (Saldırgan siparişin var olup olmadığını bilemez)
  if (!order) {
    return res.status(404).json({ error: "Sipariş bulunamadı." });
  }

  return res.json({ success: true, data: order });
});
```

💡 **Kod Analizi:** Sorguya `userId: currentUserId` koşulu eklenerek IDOR açığı kökten kapatılmıştır. Ayrıca bulunamadığında 403 yerine `404 Not Found` dönülerek saldırganın sistemdeki diğer sipariş ID'lerini taraması (Enumeration Attack) engellenmiştir.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** URL'deki ID'nin kullanıcının kendi verisi olduğunu varsaymak (IDOR).
```

  - **Çözüm:** Her veritabanı sorgusunda `WHERE id = ? AND userId = ?` sahiplik kontrolü yapın.
- **Tuzak:** React'te `dangerouslySetInnerHTML` kullanarak kullanıcı girdisini doğrudan ekrana basmak (XSS).
  - **Çözüm:** Zorunlu olmadıkça asla ham HTML render etmeyin; DOMPurify ile temizleyin.
- **Tuzak:** Güvenlik açıklarını sadece frontend tarafında gizlemeye çalışmak.
  - **Çözüm:** Frontend güvenlik sağlamaz; tüm kontroller backend tarafında yapılmalıdır.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 44: OWASP Top 10 - Web Açıklarını Kapatmak*

- 🖥️ **Slayt 1:** OWASP Top 10 nedir ve neden endüstri standardıdır?
- 🖥️ **Slayt 2:** IDOR (Yetkisiz Nesne Erişimi) açığının anatomisi ve çözümü.
- 🖥️ **Slayt 3:** XSS (Cross-Site Scripting) ve React'te güvenli render alma.
- 🖥️ **Slayt 4:** SameSite Çerezler ve CSRF saldırılarını engellemek.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Postman ile iki farklı kullanıcı token'ı oluşturun. Kullanıcı A'nın oluşturduğu siparişi, Kullanıcı B'nin token'ı ile çekmeye çalışın. Naive kodda siparişin sızdığını, sahiplik kontrolü eklenince 404 döndüğünü canlı gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki tüm `GET /resource/:id` ve `DELETE /resource/:id` endpoint'lerini inceleyin. Her birinde oturum açan kullanıcının sahiplik kontrolünün (ownership check) yapıldığını doğrulayın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: IDOR (Insecure Direct Object Reference) güvenlik açığı tam olarak nasıl meydana gelir? - A) Bilgisayarın klavyesinin bozulmasıyla - B) Uygulamanın, URL veya istek parametresindeki ID üzerinden veriyi çekerken, oturum açmış kullanıcının o veriye erişim hakkı olup olmadığını (sahiplik) denetlememesiyle - C) Sadece CSS dosyalarında - D) İnternet kablosu çekildiğinde > **Doğru Yanıt:** B > **Açıklama:** IDOR, bir kullanıcının sadece URL'deki ID numarasını (örn: 101 yerine 102 yazarak) değiştirip başkalarının verilerine erişebilmesidir.

Soru 2: React uygulamalarında kullanıcı girdilerini ekrana basarken XSS açığı oluşmasını varsayılan olarak engelleyen mekanizma nedir? - A) Otomatik HTML Entity Escaping (React süslü parantezler `{value}` içindeki tüm tehlikeli HTML karakterlerini otomatik olarak zararsız metne dönüştürür) - B)

Bilgisayarın antivirüsü - C) Sadece Google Chrome kullanılması - D) TypeScript derleyicisi > **Doğru Yanıt:** A > **Açıklama:** React, JSX içindeki değişkenleri ekrana basarken `<script>` gibi tehlikeli etiketleri otomatik olarak escape eder (`<script>`); bu sayede kod çalışamaz.